



Instituto Politécnico Nacional

Escuela Superior de Computo



Materia: Aplicaciones para Comunicaciones en Red

Profesor: Tanibet Pérez De los Santos Mondragón

Integrantes: Gil Valentín Oswaldo

Grupo: 6CV1 **Fecha:** 24/03/2026

Actividad: Ejercicio 2.4 – Chat TCP y Demultiplexación.

Ejercicio 2.4: Servidor de Chat TCP Multicliente con Demultiplexación

Desarrolla un sistema cliente-servidor donde:

- El servidor acepte múltiples conexiones TCP.
- Cada cliente se atienda en un thread independiente.
- El servidor pueda diferenciar a cada cliente (por ejemplo: IP + puerto o un nickname).
- Los mensajes enviados por un cliente se distribuyan a los demás (tipo chat).

Servidor

1. Crear un socket TCP que escuche en un puerto (ej. 5000).

```
def main():
    HOST = "192.168.56.102"
    PORT = 5050

    if len(sys.argv) == 3:
        HOST = sys.argv[1]
        PORT = int(sys.argv[2])

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as TCPServerSocket:
        # Esta opción permite reiniciar el servidor sin que el puerto se quede "atascado"
        TCPServerSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        TCPServerSocket.bind((HOST, PORT))
        TCPServerSocket.listen(10)
```

El socket TCP escucha el puerto 5050 (El que usamos normalmente en clase).

2. Por cada nueva conexión:

- Aceptar con `accept()`.

```
while True:
    client_conn, client_addr = TCPServerSocket.accept()
```

- Crear un nuevo thread para manejar ese cliente.

```
# Crear el thread independiente para el cliente recién llegado
hilo = threading.Thread(target=manejar_cliente, args=(client_conn, client_addr))
hilo.daemon = True
hilo.start()
```

3. Mantener una estructura compartida (lista o diccionario) con:

- Identificador del cliente (IP:puerto o nickname).
- Socket asociado.

```
def manejar_cliente(conn, addr):
    """Hilo independiente para cada cliente conectado."""
    try:
        # El primer mensaje que envía el cliente debe ser su nickname
        nickname = conn.recv(1024).decode('utf-8').strip()
        clientes_activos[conn] = nickname
```

Identificamos cada cliente con su IP, Puerto y el Socket asociado.

Después lo añadimos a la lista de clientes activos.

4. Implementar demultiplexación lógica:

- El servidor debe identificar de qué cliente proviene cada mensaje.

```
# Demultiplexación: Mostrar en la consola del servidor de quién viene
print(f"[Cliente {addr[0]}:{addr[1]} - {nickname}]: {mensaje}")
```

Identificamos de que cliente viene el mensaje por medio de su IP, Puerto y

Nickname, además de mostrar el mensaje que envía.

- Mostrar en consola algo como:

[Cliente 192.168.1.5:53000]: Hola

```
[Cliente 192.168.56.106:57721 - Oswaldo]: HoLA
[Cliente 192.168.56.106:57721 - Oswaldo]: Como estan chaviza?
[Cliente 192.168.56.106:57739 - Andres]: Quien libre?
[Cliente 192.168.56.106:57759 - David]: *Emoji de ojo salton*
[Cliente 192.168.56.106:57804 - Alexis]: Quieren bolis papois?
```

5. Reenviar mensajes a los demás clientes conectados.

```
def broadcast(mensaje, remitente_conn=None):
    """Envía un mensaje a todos los clientes, excepto a quien lo originó."""
    for conn in list(clientes_activos.keys()):
        if conn != remitente_conn:
            try:
                conn.sendall(mensaje.encode('utf-8'))
            except:
                # Si falla el envío, asumimos que el cliente murió y lo removemos
                conn.close()
                del clientes_activos[conn]
```

6. Manejar desconexiones:

- Eliminar al cliente de la lista.

```
del clientes_activos[conn] # Eliminar de la lista
```

- Notificar al resto.

```
broadcast(msg_salida) # Notificar al resto
```

Cliente

1. Conectarse al servidor.
2. Enviar un nickname al inicio.

```
PS C:\Users\Hideki GM\Desktop\Chat TCP> python Cliente.py
Ingresa tu nickname para entrar al chat: Oswaldo
¡Conexión exitosa! Escribe 'salir' en cualquier momento para abandonar.
```

Para facilitar las cosas primero se envía el nickname para poder identificar al cliente que entra al servidor.

3. Debe poder enviar y recibir mensajes ¿Es necesario crear dos threads?

En este caso si se crearon 2 hilos. El Hilo Principal se encarga de la configuración inicial (pedir el Nickname, conectarse y enviarlo), Arranca al hilo secundario (el daemon). Para matar el proceso después de salir de la sala, después, se queda atrapado en un ciclo infinito dedicado exclusivamente a leer el teclado y enviar mensajes al servidor.

El Hilo Secundario, su única función es escuchar. Se queda en un ciclo infinito dedicado exclusivamente a recibir mensajes del servidor y mostrarlos en la terminal del cliente.

4. Mostrar mensajes en formato:

[Juan]: Hola a todos

```
Tu mensaje:  
[Servidor]: Andres se ha unido al chat.  
Tu mensaje:  
[Servidor]: David se ha unido al chat.  
Tu mensaje:  
[Servidor]: Johan se ha unido al chat.  
Tu mensaje:  
[Servidor]: Alexis se ha unido al chat.  
Tu mensaje: HoLA  
Tu mensaje: Como estan chaviza?  
Tu mensaje:  
[Andres]: Quien libre?  
Tu mensaje:  
[David]: *Emoji de ojo salton*  
Tu mensaje:  
[Alexis]: Quieren bolis papois?  
Tu mensaje:
```